VANTARA RETAIL SOLUTIONS

CUSTOMER BEHAVIOR PREDICTION PLATFORM

CHURN | CUSTOMER VALUE | SEGMENTS | EXPLAINABLE AI

PROJECT REPORT & PORTFOLIO SHOWCASE

Retail Analytics • Machine Learning • FastAPI • Streamlit • PostgreSQL • Docker

Table of Contents

1. **Executive Summary** 3
2. **Business Problem** 3
3. **Objectives and Scope** 3
4. **Dataset and Preparation** 3
5. **Target Definition and Feature Engineering** 4
6. **Modeling and Results** 4
   - ✓ Explainable AI (SHAP and LIME) 4
   - ✓ Customer-Level Dataset 4
   - ❖ Final Model Evaluation and Reproducibility Evidence 5
   - ✓ Evaluation Protocol 5
   - ❖ Required Evaluation Results 5
   - ✓ Interpretation and Limitations 5
   - ✓ System Architecture 5
7. **Explainability and Segmentation** 6
8. **Solution Architecture** 6
9. **Streamlit User Interface** 6
10. **API and Batch Scoring** 7
11. **Acceptance Criteria Review** 7
12. **Testing and Quality Assurance** 7
13. **Security, Reliability, and Limitations** 7
14. **Reproducibility and Runbook** 8
15. **Outcomes and Portfolio Value** 8
    - ✓ Technology Stack 9
16. **Conclusion** 9
    - o **Appendix A**: Implementation Evidence and Runbook 9
    - ❖ Final Verification Commands 10
    - o **Appendix B**: Dataset Source and Attribution 20
    - o **Appendix C**: Technical References 21

1. Executive Summary

Vantara Customer Behavior Prediction Platform is an end-to-end retail analytics prototype that turns historical transaction data into customer-level insights for retention planning, customer-value assessment, and purchasing-behavior analysis. The solution combines Python data preparation and machine learning, a FastAPI prediction service, a Streamlit dashboard, a PostgreSQL service configuration, and Docker Compose deployment configuration.

The project uses the UCI Online Retail II dataset and is designed to clean invoice-line records, engineer customer-level features, estimate churn risk and customer lifetime value (CLV), explore customer segments, and expose scored outputs through an API and interactive interface. It describes a portfolio prototype; it does not claim production readiness or proven commercial uplift.

2. Business Problem

Retail teams can struggle to identify customers who are disengaging, decide which customers deserve retention attention first, and turn large transaction exports into actionable summaries. Manual reporting and fixed rules can make prioritization slow and inconsistent.

- Identify customers with elevated likelihood of not purchasing again within the defined horizon.
- Combine churn risk with estimated customer value to support prioritization.
- Summarize customer purchasing patterns using recency, frequency, monetary, and engagement indicators.
- Make outputs accessible to business users through a dashboard and downloadable files.
- Provide a reusable API interface and local deployment workflow.

The potential business benefit is better-informed retention activity. Actual reduction in churn, campaign uplift, and revenue impact has not been established through a live controlled experiment.

3. Objectives and Scope

- Build a repeatable pipeline from transaction-level records to customer-level analytical data.
- Create a churn classification workflow and a CLV estimation workflow.
- Explore customer segmentation and make outputs understandable to analysts.
- Serve supported scoring operations through FastAPI.
- Present customer metrics, risk estimates, and scored outputs through Streamlit.
- Document architecture, quality checks, deployment, limitations, and handover.

The PRD considered a broader model portfolio including Logistic Regression, Decision Tree, Random Forest, XGBoost, LightGBM, distance-based methods, ANN/LSTM/autoencoder experiments, and SHAP/LIME explanations. These are intended scope items unless the current repository contains the corresponding implementation and reproducible results.

4. Dataset and Preparation

The primary dataset is **UCI Online Retail II (dataset identifier 502)**, supplied as a workbook with two periods spanning December 2009 to December 2011. Each row represents an invoice line. Key fields include Invoice, Stock Code, Description, Quantity, Invoice Date, Unit Price, Customer Identifier, and Country.

Checkpoint	Recorded Result
Raw Transaction Rows	1,067,371
Missing Customer_ID (initial inspection)	243,007
Duplicate Rows (initial inspection)	34,335
Cleaned Transaction Rows	797,815
Rows Removed for Missing Customer ID	235,151
Cancellation Rows Removed	19,104
Invalid Quantity/Price Rows Removed	18,460
Customer-Level ML Dataset	5,281 rows × 64 features
Churn Distribution	2,989 churn / 2,292 non-churn (~56.6% churn)

The Online Retail II dataset was processed to remove duplicate records, transactions without customer identifiers, cancellation records, and invalid quantity or price values. The resulting customer-level dataset supports predictive modeling, customer segmentation, and purchasing-behavior analysis.

Note: *The figures above are recorded preprocessing results. Removal categories may overlap depending on the order of processing and exclusion rules.*

5. Target Definition and Feature Engineering

The PRD defines churn as no purchase in the next 90 days. Customer-level features are intended to summarize behavior such as recency, order frequency, spend, average order value, purchase timing, return behavior, engagement, and customer-value indicators supported by the implementation.

- **Recency**: Elapsed time since the latest purchase at the observation cutoff.
- **Frequency**: Number of orders or purchases during the observation window.
- **Monetary Indicators**: Total and average spend or related aggregates.
- **Engagement**: A composite activity indicator where implemented.
- **Purchase Probability**: Estimate exposed by the scoring interface for the defined horizon.
- **Customer Segment**: Group assignment from a clustering experiment.

Churn is defined in the project requirements as the absence of a purchase during the subsequent 90-day period. Customer-level features summarize purchasing recency, frequency, monetary value, engagement, and other behavioral indicators. The feature engineering pipeline produces a structured dataset for churn prediction, customer-value estimation, and segmentation. Consistent observation cutoffs and aligned training and inference features are important for reliable evaluation and scoring.

6. Modeling and Results

The Vantara Customer Behavior Prediction Platform combines supervised machine learning, customer segmentation, and model explainability to support retail customer analytics. Random Forest models are used for churn prediction and customer lifetime value (CLV) estimation, while K-Means and Gaussian Mixture Models are used in segmentation experiments.

Experiment/Model	Recorded Result	Interpretation
Churn Prediction — Random Forest	ROC-AUC: 0.7905 Recall: 0.7639	Recall exceeds the target of 0.70, while ROC-AUC is slightly below the target of 0.80.
CLV Estimation — Random Forest	R^2: 0.8579	Exceeds the target of 0.60.
Customer Segmentation — K-Means Clustering (k=3)	Silhouette Score: 0.2923	Highest score among the recorded clustering comparisons; separation is moderate.
Gaussian Mixture Model (k=2)	Silhouette Score: 0.2760	Lower silhouette score than K-Means with three clusters.
K-Means Clustering (k=5)	Silhouette Score: 0.2061	Lower silhouette score than K-Means with three clusters.

✓ Explainable AI (SHAP and LIME)

SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-agnostic Explanations) were used to interpret model predictions and understand the factors influencing individual customer churn risk. SHAP provides feature-contribution explanations, while LIME offers locally interpretable approximations of model predictions. Together, these techniques improve model transparency and help analysts understand the factors associated with customer-level predictions.

Recorded SHAP examples included churn probabilities of approximately 0.1519, 0.5002, and 0.8493 for low-, intermediate-, and high-risk cases. Example SHAP contributions included CLV (+0.029367), Engagement Score (−0.021781), and Recency (−0.020862). These contributions are specific to the observations and model outputs analyzed.

✓ Customer-Level Dataset

The customer-level feature engineering pipeline produced a dataset containing 5,281 customer records and 64 features. This dataset supports customer analytics, predictive modeling, and downstream segmentation.

Note: *The performance values above are the results recorded for the corresponding project models. The earlier 22-feature churn result (ROC-AUC 0.8292) and CLV Hurdle-model result (R^2 0.5871) are excluded from this final results table to avoid mixing different experiments with the dashboard's recorded model results.*

❖ Final Model Evaluation and Reproducibility Evidence

The project evaluates churn classification, customer lifetime value (CLV) estimation, customer segmentation, and model explainability. The results presented in this report represent recorded project experiments. Their attribution to the final deployed artifacts depends on verification of the model files, feature schema, test dataset, and evaluation procedure. Reproducible evaluation outputs should be retained with the submitted project to support independent validation.

✓ Evaluation Protocol

- Record the evaluation date, model artifact filename, feature set, target definition, and test split.
- Evaluate supervised models against held-out test data that was not used for training or model selection.
- Confirm that churn labels and customer features follow the defined prediction horizon and do not incorporate information from after the prediction cutoff.
- Calculate all reported metrics from the same verified evaluation run for each model.
- Preserve the evaluation output, confusion matrix, and applicable explainability plots as supporting evidence.

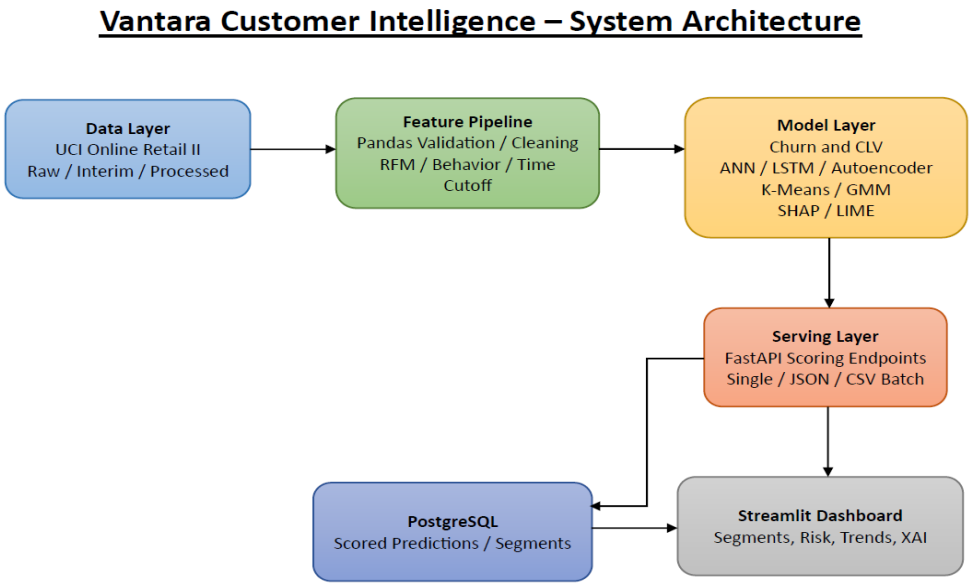
❖ Required Evaluation Results

Component	Required Evidence
Churn Classification	Accuracy, precision, recall, F1-score, ROC-AUC and confusion matrix
CLV Estimation	MAE, RMSE and R^2
Customer Segmentation	Clustering method, selected cluster count, feature set and silhouette score
Explainability	Reproducible SHAP or LIME output for the evaluated model, where implemented

✓ Interpretation and Limitations

Previously recorded experiment results are retained as historical evidence until they have been reproduced against the final submitted artifacts. Historical metrics must not be represented as the performance of the deployed model without verifying artifact identity, feature alignment and test-data provenance.

✓ System Architecture



The architecture illustrates the flow from retail transaction data preparation and customer-level feature engineering to trained model artifacts, prediction services, persistence components, and the Streamlit dashboard. The FastAPI service exposes supported scoring operations, while Docker Compose coordinates the configured application services.

7. Explainability and Segmentation

The project aims to provide more than a prediction label: analysts should be able to inspect risk estimates and, where implemented, understand influential features. Prior exploratory work used SHAP 0.52.0 and recorded illustrative churn probabilities of approximately 0.1519, 0.5002, and 0.8493 for low-, borderline-, and high-risk examples.

Recorded SHAP contributions in one example included CLV (+0.029367), Engagement Score (−0.021781), and Recency (−0.020862). These are case-specific values; the direction and magnitude depend on the explained output, baseline, and observation. They should not be presented as universal rules.

Clustering experiments compared K-Means and Gaussian Mixture Models. Segment labels should only be assigned business meanings after reviewing actual cluster summaries. A silhouette score alone does not prove that segments are stable or commercially actionable.

8. Solution Architecture

Component	Responsibility
Data Processing	Cleans transaction records and generates customer-level analytical features.
Model Artifacts	Stores the trained churn and CLV models used for scoring.
FastAPI	Provides prediction endpoints, request validation, health checks, and interactive API documentation.
Streamlit Dashboard	Presents customer behavior metrics, churn risk, CLV estimates, segmentation results, and available exports.
PostgreSQL 16	Provides the configured relational database service for project data persistence.
Docker Compose	Defines the local application services and their configuration.

The application uses **port 8000 for FastAPI** and **port 8501 for Streamlit**. API documentation is available at **http://127.0.0.1:8000/docs**, and the dashboard is available at **http://localhost:8501** when the corresponding services are running.

9. Streamlit User Interface

The Streamlit dashboard is the principal demonstration surface for non-technical stakeholders. Across prior iterations, the Vantara interface has been designed around customer spending and purchasing behavior, churn risk, CLV estimates, customer records, and scored/priority CSV outputs.

- **Overview**: Customer-level behavioral indicators and summary measures.
- **Risk Review**: Churn predictions and probabilities where present in the loaded output.
- **Value and Segment Exploration**: Estimated CLV and customer groups were supported by the current build.
- **Scored Records**: Inspect customer-level output fields.
- **Export Workflow**: Download scored or priority customer lists when enabled in the current UI.

The most recently referenced project root is `**customer-behavior-prediction**`, with `**frontend/dashboard.py**` as the Streamlit entry point.

Recommended Evidence: Overview, Risk Table, Segment/Value View, Successful CSV Output, and API Docs/Health Check.

10. API and Batch Scoring

The FastAPI service exposes customer-scoring functionality and supports CSV-based batch scoring. A previously recorded successful request returned HTTP 200 with a CSV response containing the following fields:

- Customer_ID
- Churn_Prediction
- Churn_Probability
- 90_Day_Purchase_Probability
- Predicted_CLV

These outputs allow customer-level risk and estimated value to be reviewed or exported for further analysis. API reliability and performance should be interpreted in the context of the request type, dataset size, and test environment used for measurement.

The PRD specifies a p95 API latency target below 400 ms. The recorded 268.94 ms result should be described as the measured latency for the specific benchmark that produced it, rather than as a universal end-to-end performance guarantee.

11. Acceptance Criteria Review

Criterion	PRD Target	Recorded Result / Status
Churn ROC-AUC	≥ 0.80	0.7905
Churn Recall	≥ 0.70	0.7639 (76.39%)
CLV R^2	≥ 0.60	0.8579
API p95 Latency	< 400 ms	268.94 ms
Dashboard Performance	< 3 seconds	1.06 seconds
Automated Test Coverage	≥ 70%	88% Coverage (16 tests passed)
Docker Compose	Single-command local launch	Configuration prepared
Handover Deliverables	README, diagrams, API docs, walkthrough, project report	Report and documentation deliverables

The platform demonstrates the main workflow for retail customer analytics, including churn prediction, customer-value estimation, segmentation experiments, API-based scoring, and dashboard visualization. The churn ROC-AUC result is below the specified target, while the recorded recall and CLV R^2 exceed their respective targets.

12. Testing and Quality Assurance

The project uses pytest for automated testing and coverage measurement. The recorded test summary reports 16 tests passed and 88% code coverage.

Testing activities cover the following areas:

- **Data Processing**: Validation of feature transformations, missing values, invalid records, and output schemas.
- **API Functionality**: Health checks, valid scoring requests, CSV responses, malformed inputs, and error handling.
- **Model Integration**: Model loading, expected prediction fields, finite outputs, and valid probability ranges.
- **Data Integrity**: Duplicate handling, cancellation filtering, missing customer identifiers, and feature consistency.
- **Deployment**: Application configuration, service startup, and connectivity between configured components.

Automated tests help assess implementation reliability. They do not, by themselves, establish predictive accuracy, business impact, or production readiness.

13. Security, Reliability, and Limitations

- Keep database credentials and secrets outside source control; use env. variables and an ignored local `**.env**` file.
- Use public or synthetic customer examples in portfolio screenshots.
- Treat predictions as decision support rather than automatic grounds to exclude customers from offers.
- Document training period, prediction horizon, temporal split, and possible seasonality or drift.
- Before production, add appropriate access controls, monitoring, resource limits, deployment hardening, and retention controls.

Important limitations include the public dataset's age and scope, the gap between retrospective metrics and live campaign outcomes, potential temporal leakage if cutoffs are incorrect, uncertainty in CLV estimates, and moderate separation in the recorded clusters. Do not claim product recommendation or anomaly detection as complete without direct implementation evidence.

14. Reproducibility and Runbook

- **Project Directory**: C:\Users\RITUL\OneDrive\Desktop\AI Projects\customer-behavior-prediction
- **Main Dashboard**: frontend\dashboard.py
- **Service Endpoints**:
  - **Streamlit Dashboard**: http://localhost:8501
  - **FastAPI Documentation**: http://127.0.0.1:8000/docs
  - **FastAPI Health Check**: http://127.0.0.1:8000/health

Local Execution:

1. Open PowerShell in the project root.
2. Validate the Docker Compose configuration:
   docker compose config
3. Build and start the services:
   docker compose up -d --build
4. Check service status:
   docker compose ps -a
5. Inspect service logs:
   docker compose logs --tail=100
6. Check API health:
   Invoke-RestMethod http://127.0.0.1:8000/health
7. Check Streamlit health:
   Invoke-RestMethod http://127.0.0.1:8501/_stcore/health
8. Run automated tests with coverage:
   python -m pytest --cov
9. Open the API documentation and dashboard using the URLs listed above.

The exact startup requirements and environment variables are defined by the project's Docker Compose configuration and README.md file.

15. Outcomes and Portfolio Value

The Vantara Customer Behavior Prediction Platform demonstrates an end-to-end retail analytics workflow covering transaction data preparation, customer-level feature engineering, predictive modeling, customer segmentation, model explainability, API-based scoring, interactive dashboard development, and containerized application configuration.

The recorded model results show a churn ROC-AUC of 0.7905 and recall of 0.7639. Recall exceeds the PRD target, while ROC-AUC remains slightly below the required threshold. The recorded CLV model R^2 of 0.8579 exceeds its target of 0.60. K-Means clustering with three clusters achieved a recorded silhouette score of 0.2923, indicating moderate separation.

The project provides a practical portfolio demonstration of how machine learning outputs can be integrated into an interface designed to support customer-risk review and customer-value analysis. Actual churn reduction, revenue growth, and campaign uplift have not been established through a live controlled business experiment.

✓ Technology Stack

Area	Technology / Artifact
Data Analysis	Python, Pandas, NumPy
Machine Learning	Scikit-learn, Random Forest
Customer Segmentation	K-Means, Gaussian Mixture Models
API	FastAPI, Uvicorn, Pydantic
Dashboard	Streamlit
Database	PostgreSQL
Deployment	Docker, Docker Compose
Testing	Pytest, coverage tooling
Explainability	SHAP, LIME
Dataset	UCI Online Retail II, dataset 502

16. Conclusion

The Vantara Customer Behavior Prediction Platform integrates retail transaction processing, customer-level feature engineering, machine learning, customer segmentation, API-based scoring, and an interactive Streamlit dashboard into a unified portfolio prototype.

The project demonstrates the practical application of predictive analytics to customer-retention planning and customer-value assessment. Its recorded results indicate that the CLV model exceeds its target and churn recall meets its target, while churn ROC-AUC remains an area for improvement.

The platform also demonstrates the integration of backend services, database configuration, containerization, testing, and technical documentation. The solution provides a foundation for further improvements in model evaluation, operational monitoring, and deployment reliability.

○ Appendix A: Implementation Evidence and Runbook

This appendix presents supporting evidence for the Vantara Customer Behavior Prediction Platform, including dashboard functionality, API integration, batch scoring, database connectivity, automated testing, and local deployment configuration.

The evidence covers:

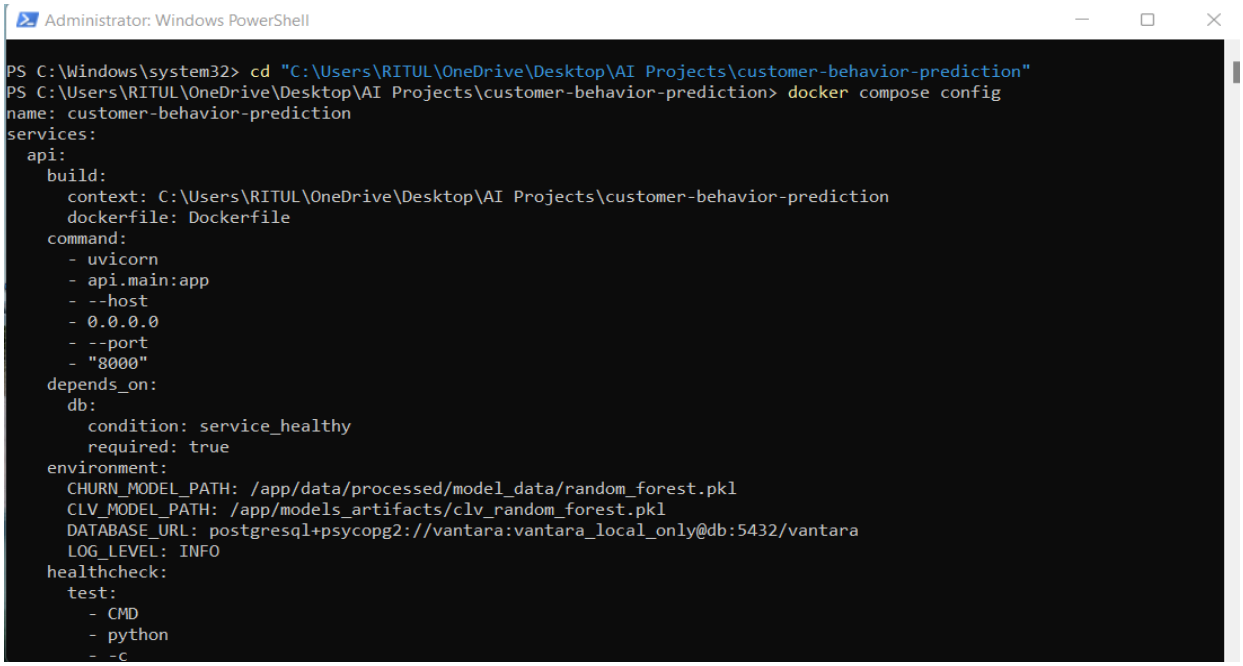
1. **Dashboard Overview**: Customer metrics, purchasing behavior, and summary visualizations.
2. **Churn Prediction**: Customer-level churn predictions and risk probabilities.
3. **Customer Lifetime Value and Segmentation**: Available CLV estimates and customer-group visualizations.
4. **API Documentation and Health Check**: FastAPI documentation and health endpoint responses.
5. **Batch CSV Scoring**: Sample scoring request and returned CSV output fields.
6. **Deployment and Testing**: Docker Compose service status, database readiness, and automated test results.

❖ Final Verification Commands

The following commands are intended for execution in Windows PowerShell from the final project root. Capture the resulting terminal output for the project evidence folder.

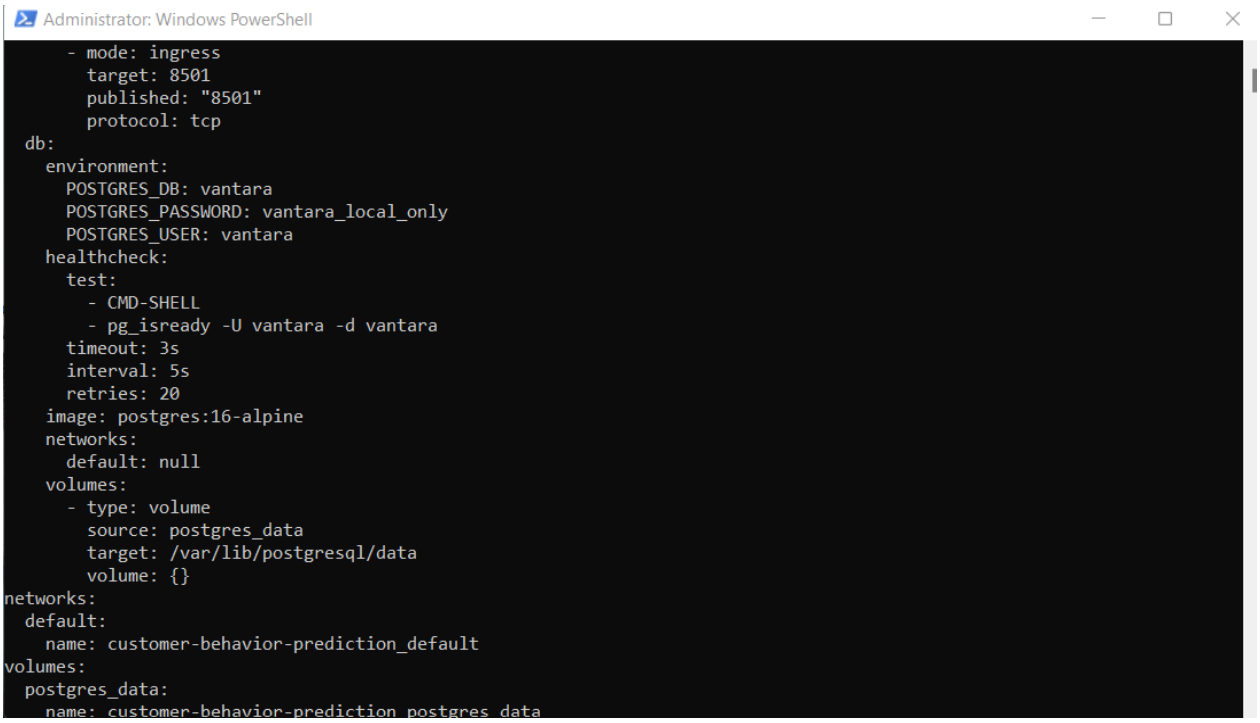
1. **Navigate to the Project Directory**

   cd "C:\Users\RITUL\OneDrive\Desktop\AI Projects\customer-behavior-prediction"



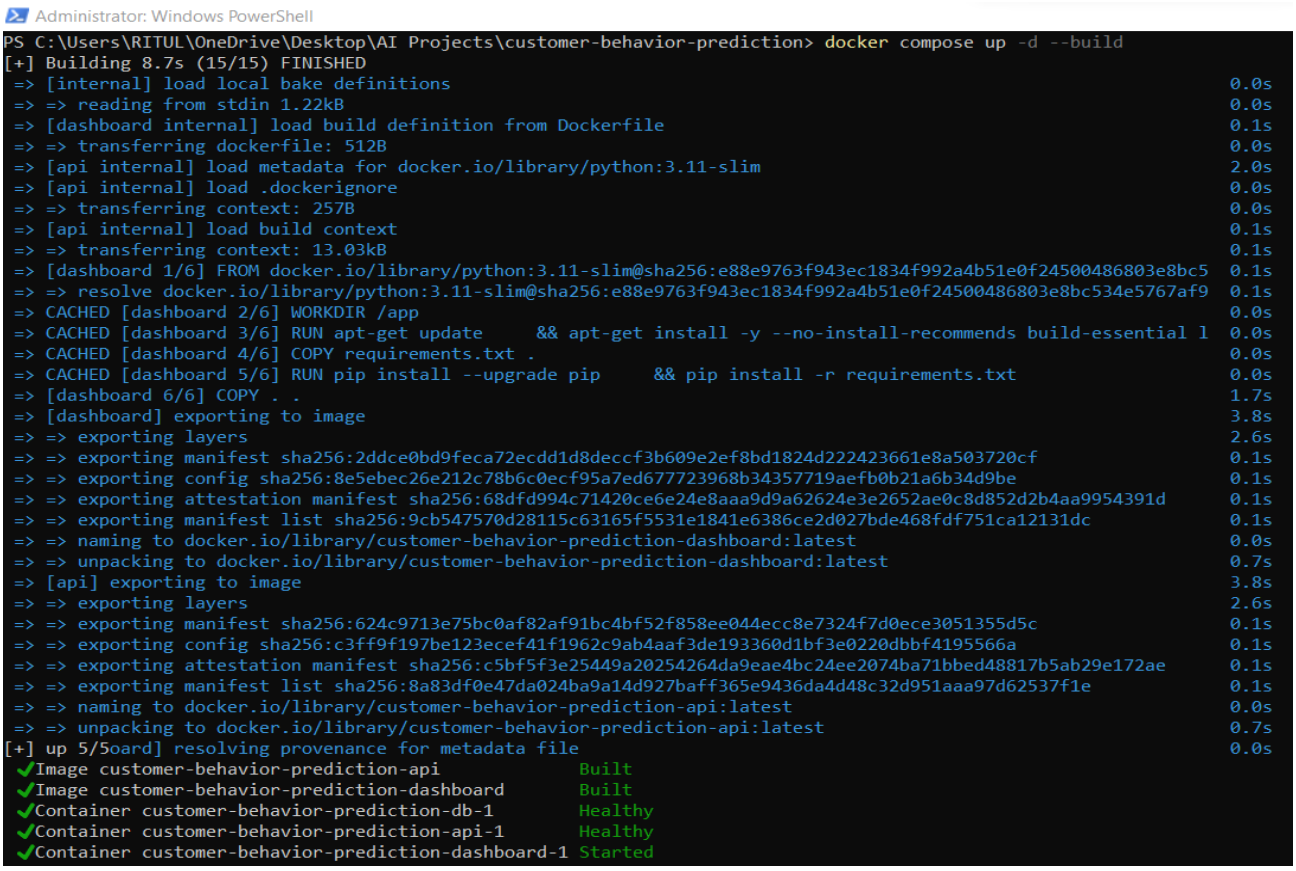
2. **Validate the Docker Compose Configuration**

   docker compose config



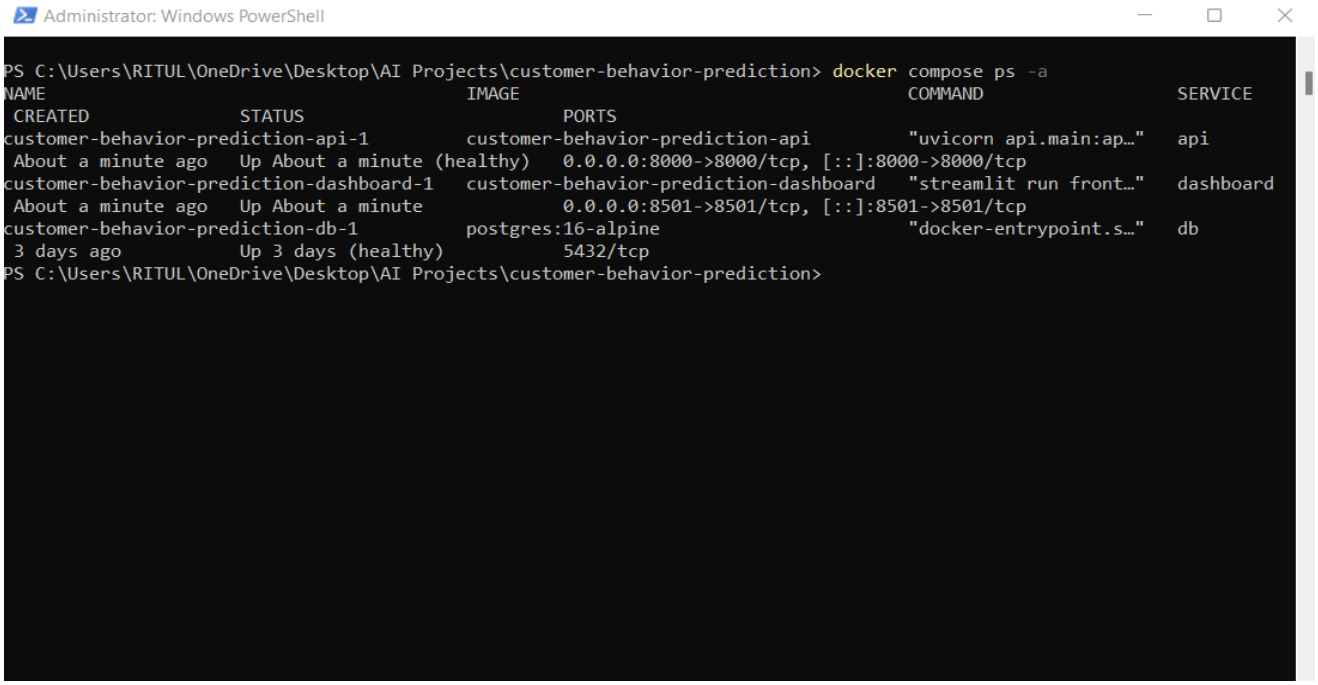
3. **Start the Application Services**

docker compose up -d --build



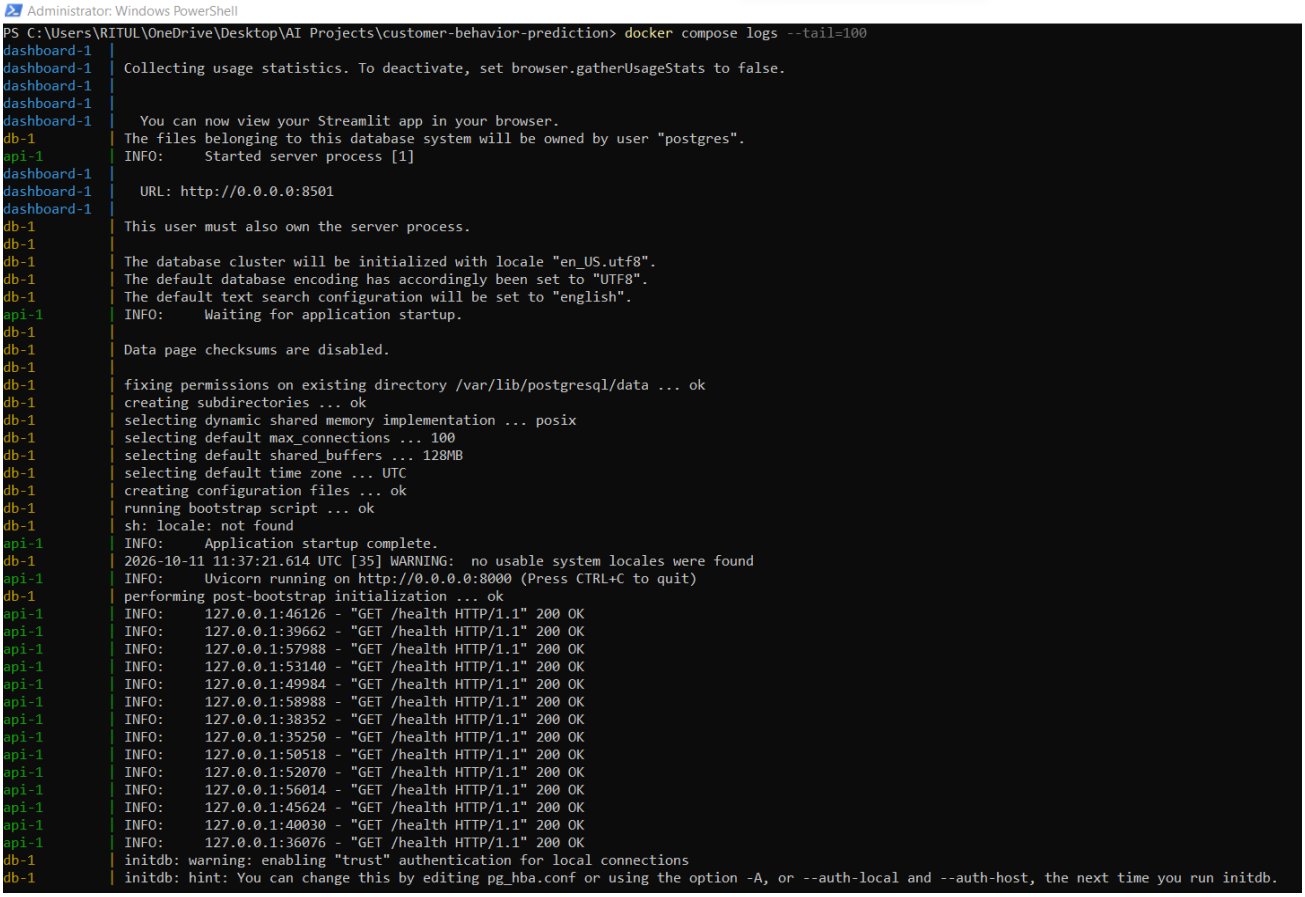
4. **Check the Service Status**

docker compose ps -a

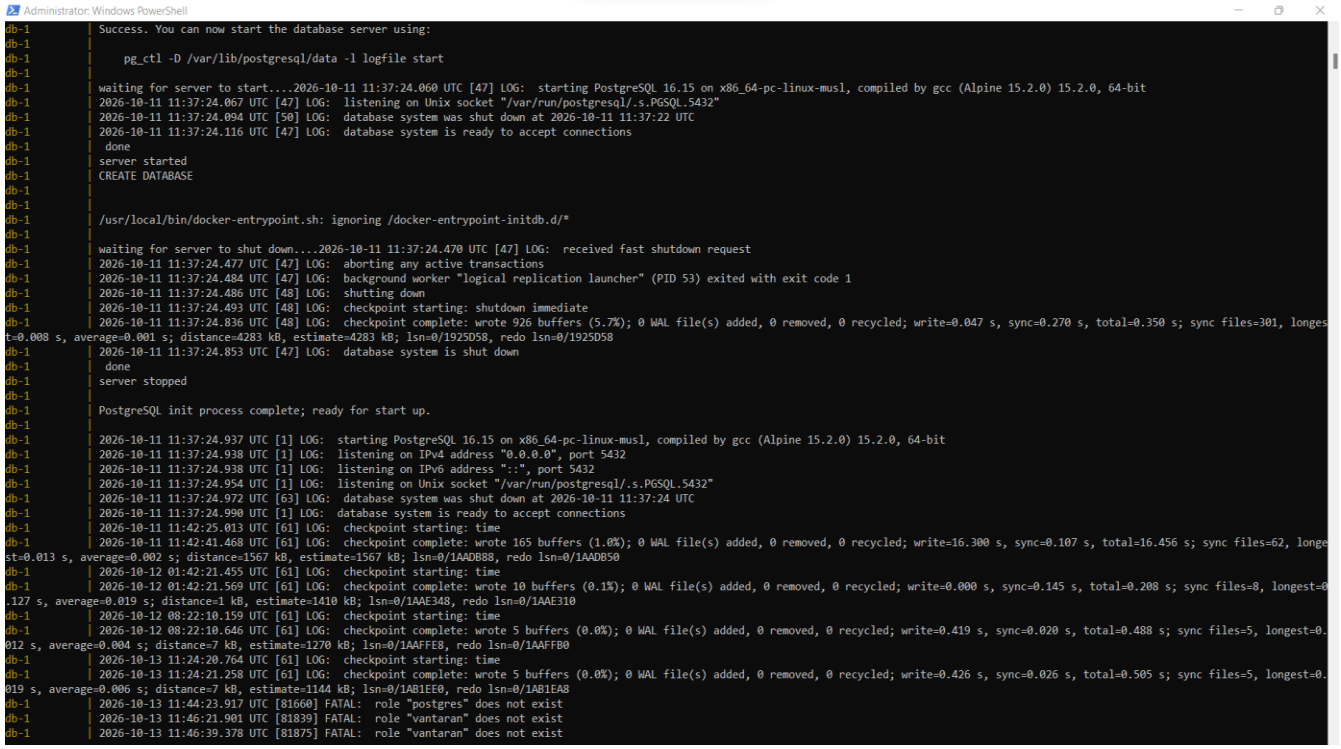


5. Inspect the Latest Service Logs

docker compose logs --tail=100



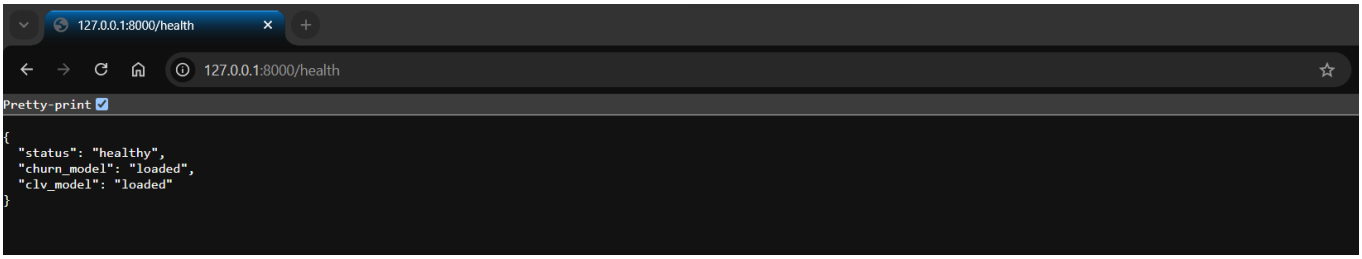
```
Administrator: Windows PowerShell
PS C:\Users\RITUL\OneDrive\Desktop\AI Projects\customer-behavior-prediction> docker compose logs --tail=100
dashboard-1  |
dashboard-1  | Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.
dashboard-1  |
dashboard-1  |
dashboard-1  |   You can now view your Streamlit app in your browser.
db-1         | The files belonging to this database system will be owned by user "postgres".
api-1        | INFO:     Started server process [1]
dashboard-1  |
dashboard-1  |   URL: http://0.0.0.0:8501
dashboard-1  |
db-1         | This user must also own the server process.
db-1         |
db-1         | The database cluster will be initialized with locale "en_US.utf8".
db-1         | The default database encoding has accordingly been set to "UTF8".
db-1         | The default text search configuration will be set to "english".
api-1        | INFO:     Waiting for application startup.
db-1         |
db-1         | Data page checksums are disabled.
db-1         |
db-1         | fixing permissions on existing directory /var/lib/postgresql/data ... ok
db-1         | creating subdirectories ... ok
db-1         | selecting dynamic shared memory implementation ... posix
db-1         | selecting default max_connections ... 100
db-1         | selecting default shared_buffers ... 128MB
db-1         | selecting default time zone ... UTC
db-1         | creating configuration files ... ok
db-1         | running bootstrap script ... ok
db-1         | sh: locale: not found
api-1        | INFO:     Application startup complete.
db-1         | 2026-10-11 11:37:21.614 UTC [35] WARNING:  no usable system locales were found
api-1        | INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
db-1         | performing post-bootstrap initialization ... ok
api-1        | INFO:     127.0.0.1:46126 - "GET /health HTTP/1.1" 200 OK
api-1        | INFO:     127.0.0.1:39662 - "GET /health HTTP/1.1" 200 OK
api-1        | INFO:     127.0.0.1:57988 - "GET /health HTTP/1.1" 200 OK
api-1        | INFO:     127.0.0.1:53140 - "GET /health HTTP/1.1" 200 OK
api-1        | INFO:     127.0.0.1:49984 - "GET /health HTTP/1.1" 200 OK
api-1        | INFO:     127.0.0.1:58988 - "GET /health HTTP/1.1" 200 OK
api-1        | INFO:     127.0.0.1:38352 - "GET /health HTTP/1.1" 200 OK
api-1        | INFO:     127.0.0.1:35250 - "GET /health HTTP/1.1" 200 OK
api-1        | INFO:     127.0.0.1:50518 - "GET /health HTTP/1.1" 200 OK
api-1        | INFO:     127.0.0.1:52070 - "GET /health HTTP/1.1" 200 OK
api-1        | INFO:     127.0.0.1:56014 - "GET /health HTTP/1.1" 200 OK
api-1        | INFO:     127.0.0.1:45624 - "GET /health HTTP/1.1" 200 OK
api-1        | INFO:     127.0.0.1:40030 - "GET /health HTTP/1.1" 200 OK
api-1        | INFO:     127.0.0.1:36076 - "GET /health HTTP/1.1" 200 OK
db-1         | initdb: warning: enabling "trust" authentication for local connections
db-1         | initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host, the next time you run initdb.
```



```
Administrator: Windows PowerShell
db-1         | Success. You can now start the database server using:
db-1         |
db-1         |     pg_ctl -D /var/lib/postgresql/data -l logfile start
db-1         |
db-1         | waiting for server to start....2026-10-11 11:37:24.060 UTC [47] LOG:  starting PostgreSQL 16.15 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
db-1         | 2026-10-11 11:37:24.067 UTC [47] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-1         | 2026-10-11 11:37:24.094 UTC [50] LOG:  database system was shut down at 2026-10-11 11:37:22 UTC
db-1         | 2026-10-11 11:37:24.116 UTC [47] LOG:  database system is ready to accept connections
db-1         |  done
db-1         | server started
db-1         | CREATE DATABASE
db-1         |
db-1         |
db-1         | /usr/local/bin/docker-entrypoint.sh: ignoring /docker-entrypoint-initdb.d/*
db-1         |
db-1         | waiting for server to shut down....2026-10-11 11:37:24.470 UTC [47] LOG:  received fast shutdown request
db-1         | 2026-10-11 11:37:24.477 UTC [47] LOG:  aborting any active transactions
db-1         | 2026-10-11 11:37:24.484 UTC [47] LOG:  background worker "logical replication launcher" (PID 53) exited with exit code 1
db-1         | 2026-10-11 11:37:24.486 UTC [48] LOG:  shutting down
db-1         | 2026-10-11 11:37:24.493 UTC [48] LOG:  checkpoint starting: shutdown immediate
db-1         | 2026-10-11 11:37:24.836 UTC [48] LOG:  checkpoint complete: wrote 926 buffers (5.7%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.047 s, sync=0.270 s, total=0.350 s; sync files=301, longes
t=0.008 s, average=0.001 s; distance=4283 kB, estimate=4283 kB; lsn=0/1925D58, redo lsn=0/1925D58
db-1         | 2026-10-11 11:37:24.853 UTC [47] LOG:  database system is shut down
db-1         |  done
db-1         | server stopped
db-1         |
db-1         | PostgreSQL init process complete; ready for start up.
db-1         |
db-1         | 2026-10-11 11:37:24.937 UTC [1] LOG:  starting PostgreSQL 16.15 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
db-1         | 2026-10-11 11:37:24.938 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
db-1         | 2026-10-11 11:37:24.938 UTC [1] LOG:  listening on IPv6 address "::", port 5432
db-1         | 2026-10-11 11:37:24.954 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-1         | 2026-10-11 11:37:24.972 UTC [63] LOG:  database system was shut down at 2026-10-11 11:37:24 UTC
db-1         | 2026-10-11 11:37:24.990 UTC [1] LOG:  database system is ready to accept connections
db-1         | 2026-10-11 11:42:25.013 UTC [61] LOG:  checkpoint starting: time
db-1         | 2026-10-11 11:42:41.468 UTC [61] LOG:  checkpoint complete: wrote 165 buffers (1.0%); 0 WAL file(s) added, 0 removed, 0 recycled; write=16.300 s, sync=0.107 s, total=16.456 s; sync files=62, longe
st=0.013 s, average=0.002 s; distance=1567 kB, estimate=1567 kB; lsn=0/1AADB88, redo lsn=0/1AADB50
db-1         | 2026-10-12 01:42:21.455 UTC [61] LOG:  checkpoint starting: time
db-1         | 2026-10-12 01:42:21.569 UTC [61] LOG:  checkpoint complete: wrote 10 buffers (0.1%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.000 s, sync=0.145 s, total=0.208 s; sync files=8, longest=0
.127 s, average=0.019 s; distance=1 kB, estimate=1410 kB; lsn=0/1AAE348, redo lsn=0/1AAE310
db-1         | 2026-10-12 08:22:10.159 UTC [61] LOG:  checkpoint starting: time
db-1         | 2026-10-12 08:22:10.646 UTC [61] LOG:  checkpoint complete: wrote 5 buffers (0.0%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.419 s, sync=0.020 s, total=0.488 s; sync files=5, longest=0.
012 s, average=0.004 s; distance=7 kB, estimate=1270 kB; lsn=0/1AAFFE8, redo lsn=0/1AAFFB0
db-1         | 2026-10-13 11:24:20.764 UTC [61] LOG:  checkpoint starting: time
db-1         | 2026-10-13 11:24:21.258 UTC [61] LOG:  checkpoint complete: wrote 5 buffers (0.0%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.426 s, sync=0.026 s, total=0.505 s; sync files=5, longest=0.
019 s, average=0.006 s; distance=7 kB, estimate=1144 kB; lsn=0/1AB1EE0, redo lsn=0/1AB1EA8
db-1         | 2026-10-13 11:44:23.917 UTC [81660] FATAL:  role "postgres" does not exist
db-1         | 2026-10-13 11:46:21.901 UTC [81839] FATAL:  role "vantaran" does not exist
db-1         | 2026-10-13 11:46:39.378 UTC [81875] FATAL:  role "vantaran" does not exist
```

6. **Verify the API Health Endpoint**

Invoke-RestMethod http://127.0.0.1:8000/health



7. **Verify the Streamlit Dashboard Health Endpoint**

Invoke-RestMethod http://127.0.0.1:8501/_stcore/health

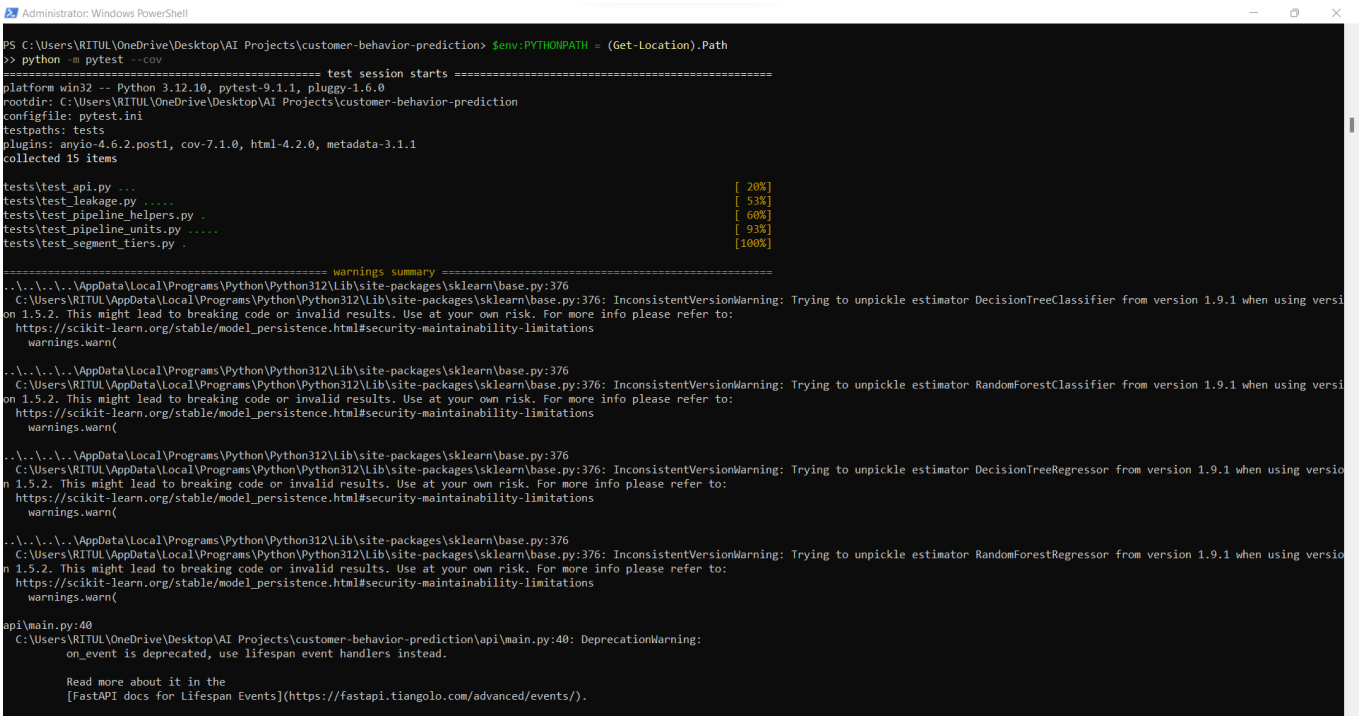


ok

8. **Run the Automated Tests with Coverage**

$env:PYTHONPATH = (Get-Location).Path

python -m pytest --cov



```
Administrator: Windows PowerShell

   @app.on_event("startup")

..\..\..\..\AppData\Local\Programs\Python\Python312\Lib\site-packages\fastapi\applications.py:4495
  C:\Users\RITUL\AppData\Local\Programs\Python\Python312\Lib\site-packages\fastapi\applications.py:4495: DeprecationWarning:
        on_event is deprecated, use lifespan event handlers instead.

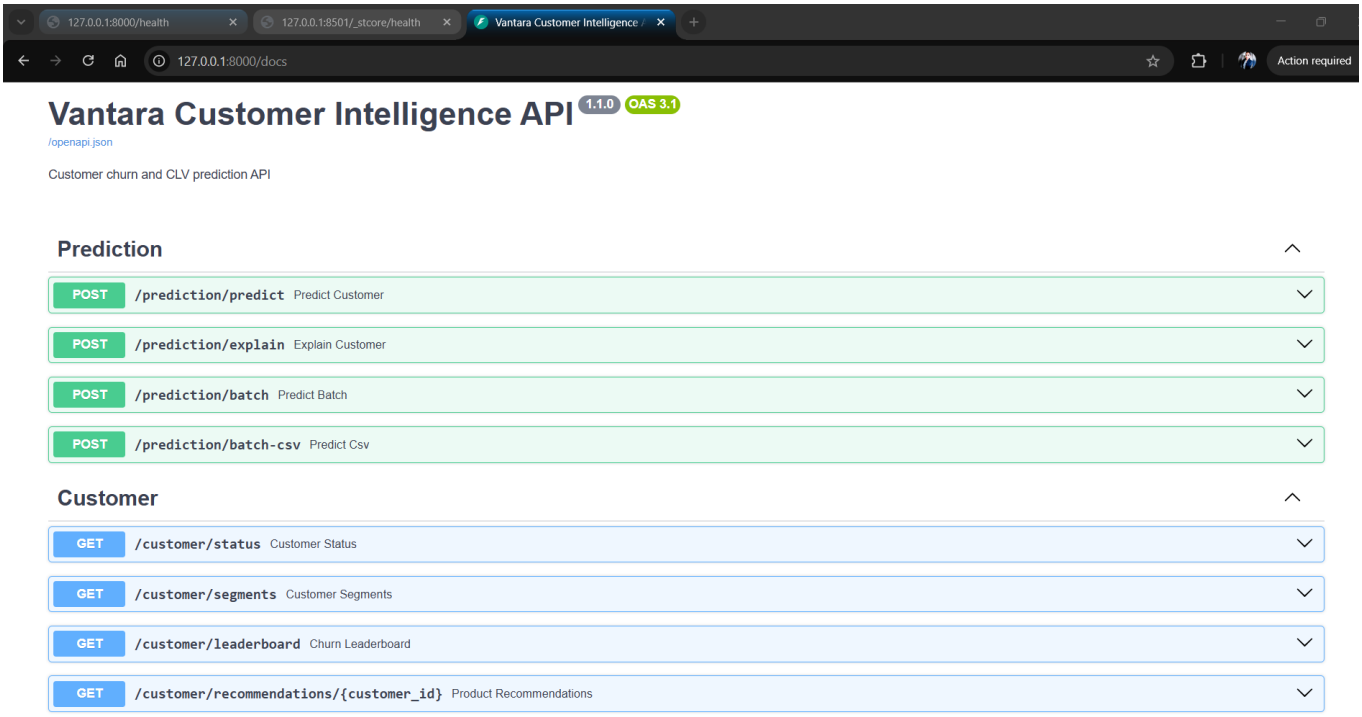
        Read more about it in the
        [FastAPI docs for Lifespan Events](https://fastapi.tiangolo.com/advanced/events/).

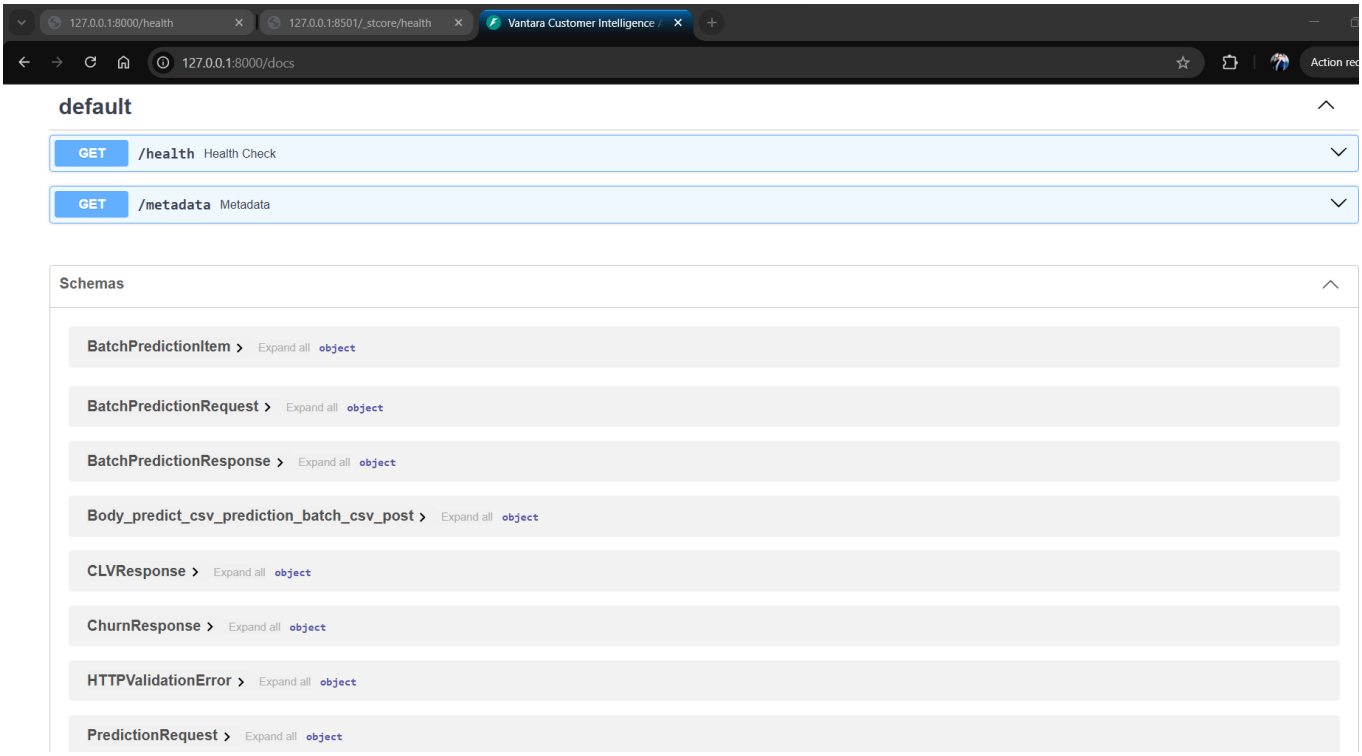
   return self.router.on_event(event_type)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================================ tests coverage ================================================
_____________________________________ coverage: platform win32, python 3.12.10-final-0 _____________________________________

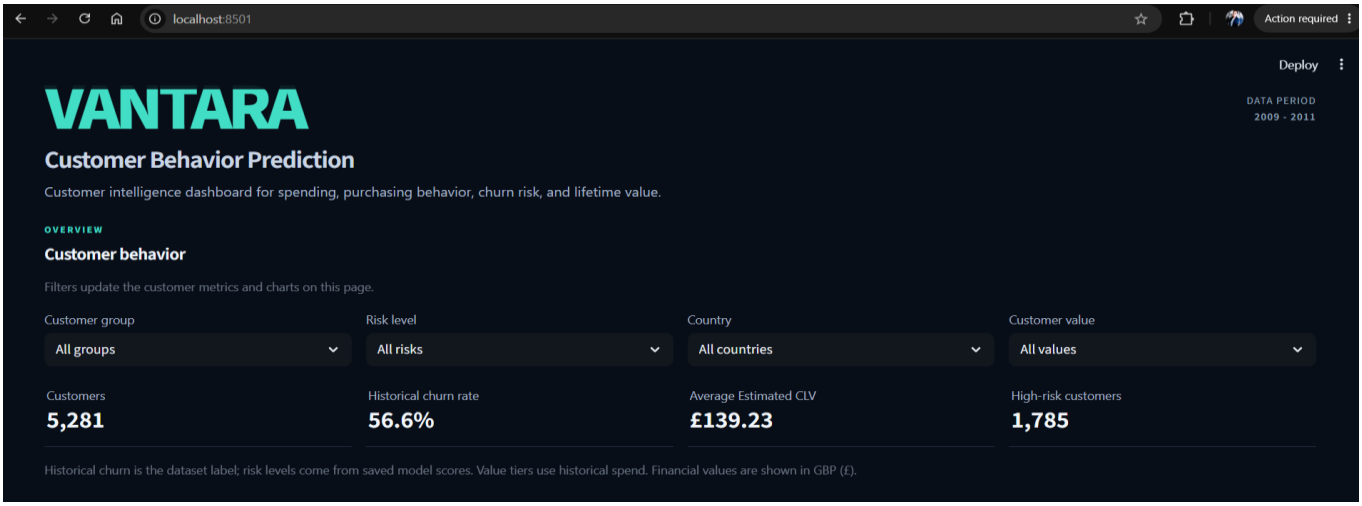
Name                                Stmts   Miss  Cover
-------------------------------------------------------
api\database.py                        95     16    83%
api\main.py                            44      7    84%
api\routers\customer.py                20      5    75%
api\routers\prediction.py             112     81    28%
api\schemas\request.py                  8      0   100%
api\schemas\response.py                22      0   100%
api\settings.py                        10      0   100%
src\__init__.py                         0      0   100%
src\data\__init__.py                    0      0   100%
src\data\clean_data.py                 55      7    87%
src\data\load_data.py                  32      1    97%
src\data\validation.py                 45      5    89%
src\pipeline.py                        23      5    78%
src\recommendations.py                 26      5    81%
src\utils\config.py                    15      1    93%
src\utils\segment_tiers.py             12      1    92%
tests\test_api.py                      24      0   100%
tests\test_leakage.py                  36      0   100%
tests\test_pipeline_helpers.py          6      0   100%
tests\test_pipeline_units.py           67      0   100%
tests\test_segment_tiers.py             7      0   100%
-------------------------------------------------------
TOTAL                                 659    134    80%
========================================== 15 passed, 6 warnings in 39.84s ==========================================
PS C:\Users\RITUL\OneDrive\Desktop\AI Projects\customer-behavior-prediction>
```

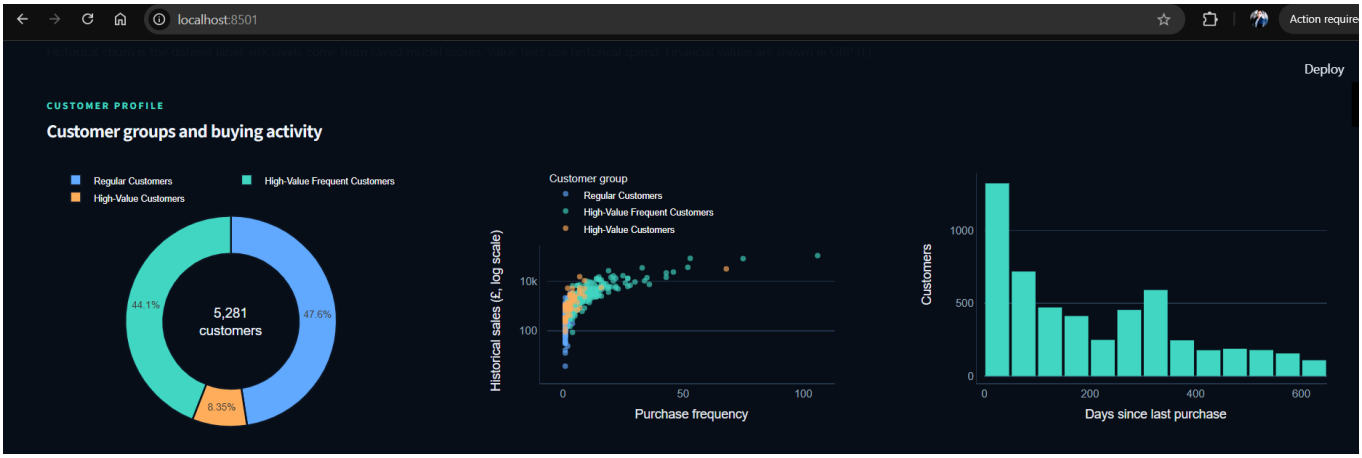
9. **Review the API Documentation**
   - Start the services, then open the following URL in a browser:
   - http://127.0.0.1:8000/docs

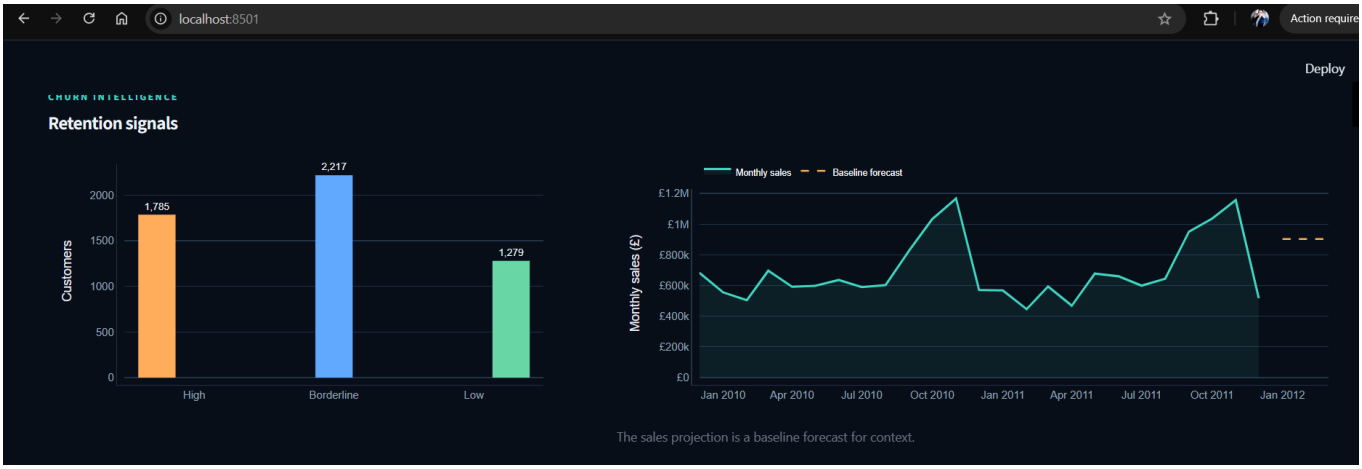


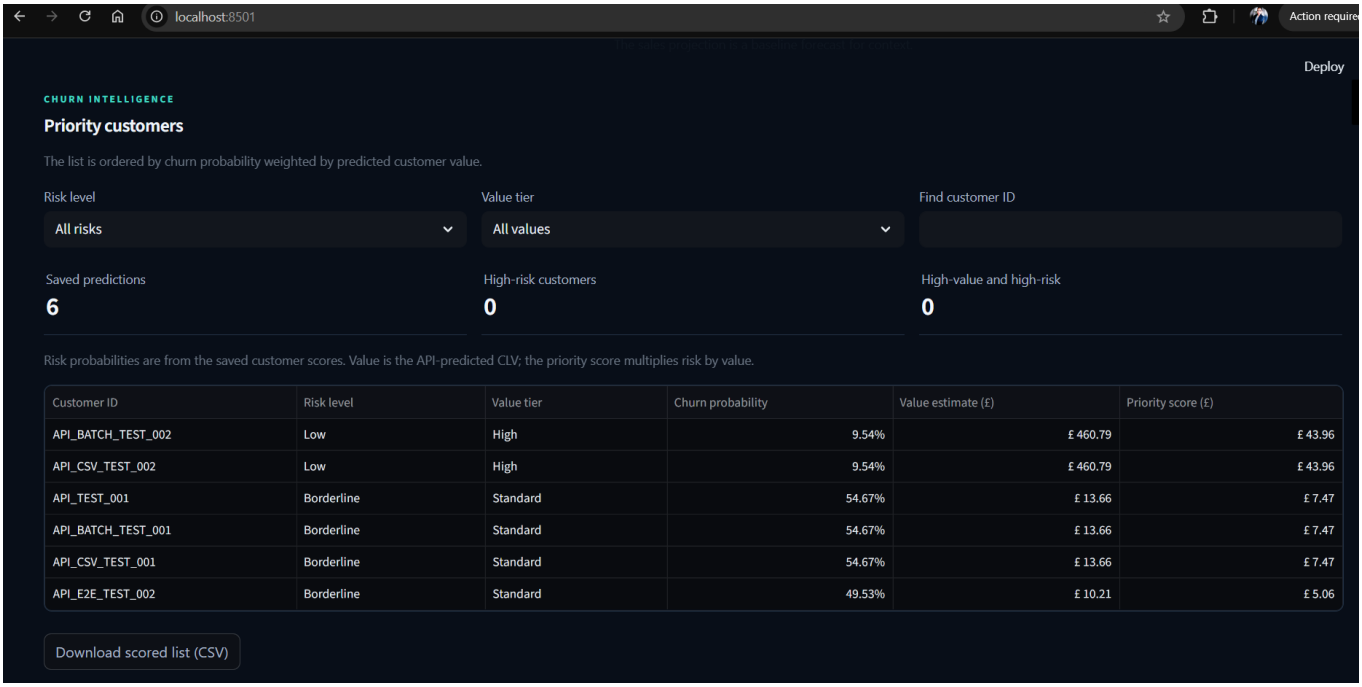


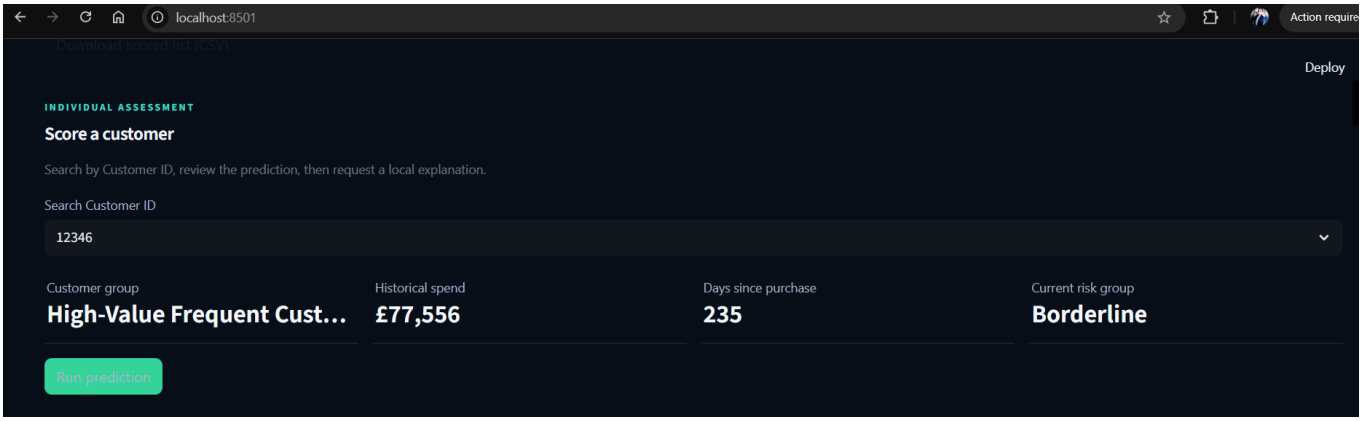
10. **Open the Dashboard**
    - http://localhost:8501






localhost:8501
Action require
Deploy
CHURN INTELLIGENCE
Retention signals
Customers
2000
1500
1000
500
0
1,785
2,217
1,279
High
Borderline
Low
Monthly sales
Baseline forecast
Monthly sales (£)
£1.2M
£1M
£800k
£600k
£400k
£200k
£0
Jan 2010
Apr 2010
Jul 2010
Oct 2010
Jan 2011
Apr 2011
Jul 2011
Oct 2011
Jan 2012
The sales projection is a baseline forecast for context.

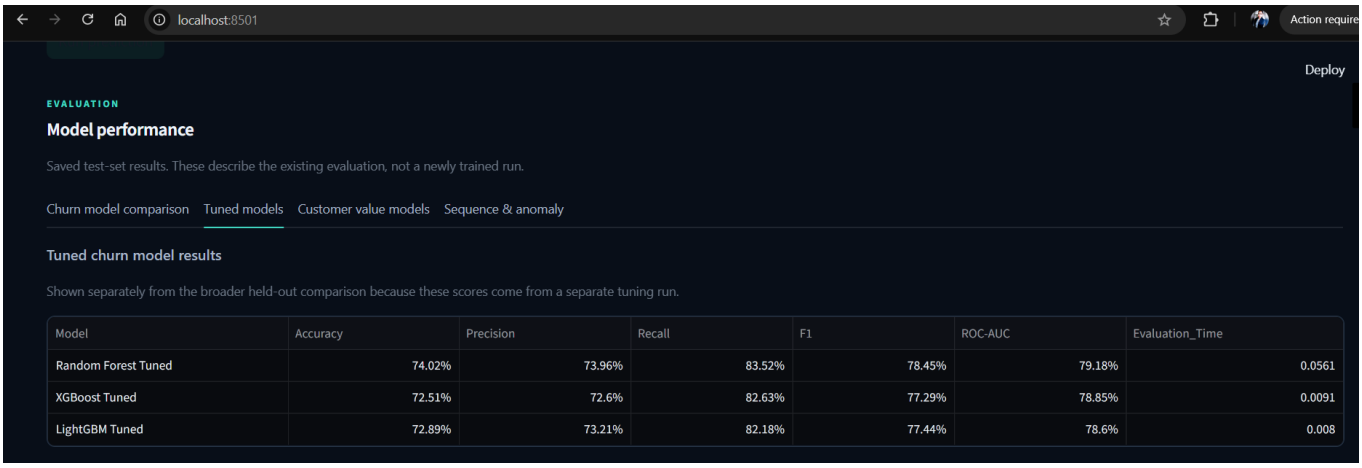

localhost:8501
Action require
Deploy
CHURN INTELLIGENCE
Priority customers
The list is ordered by churn probability weighted by predicted customer value.
Risk level
All risks
Value tier
All values
Find customer ID
Saved predictions
6
High-risk customers
0
High-value and high-risk
0
Risk probabilities are from the saved customer scores. Value is the API-predicted CLV; the priority score multiplies risk by value.
Customer ID | Risk level | Value tier | Churn probability | Value estimate (£) | Priority score (£)
API_BATCH_TEST_002 | Low | High | 9.54% | £ 460.79 | £ 43.96
API_CSV_TEST_002 | Low | High | 9.54% | £ 460.79 | £ 43.96
API_TEST_001 | Borderline | Standard | 54.67% | £ 13.66 | £ 7.47
API_BATCH_TEST_001 | Borderline | Standard | 54.67% | £ 13.66 | £ 7.47
API_CSV_TEST_001 | Borderline | Standard | 54.67% | £ 13.66 | £ 7.47
API_E2E_TEST_002 | Borderline | Standard | 49.53% | £ 10.21 | £ 5.06
Download scored list (CSV)


localhost:8501
Action require
Deploy
INDIVIDUAL ASSESSMENT
Score a customer
Search by Customer ID, review the prediction, then request a local explanation.
Search Customer ID
12346
Customer group
High-Value Frequent Cust...
Historical spend
£77,556
Days since purchase
235
Current risk group
Borderline
Run prediction

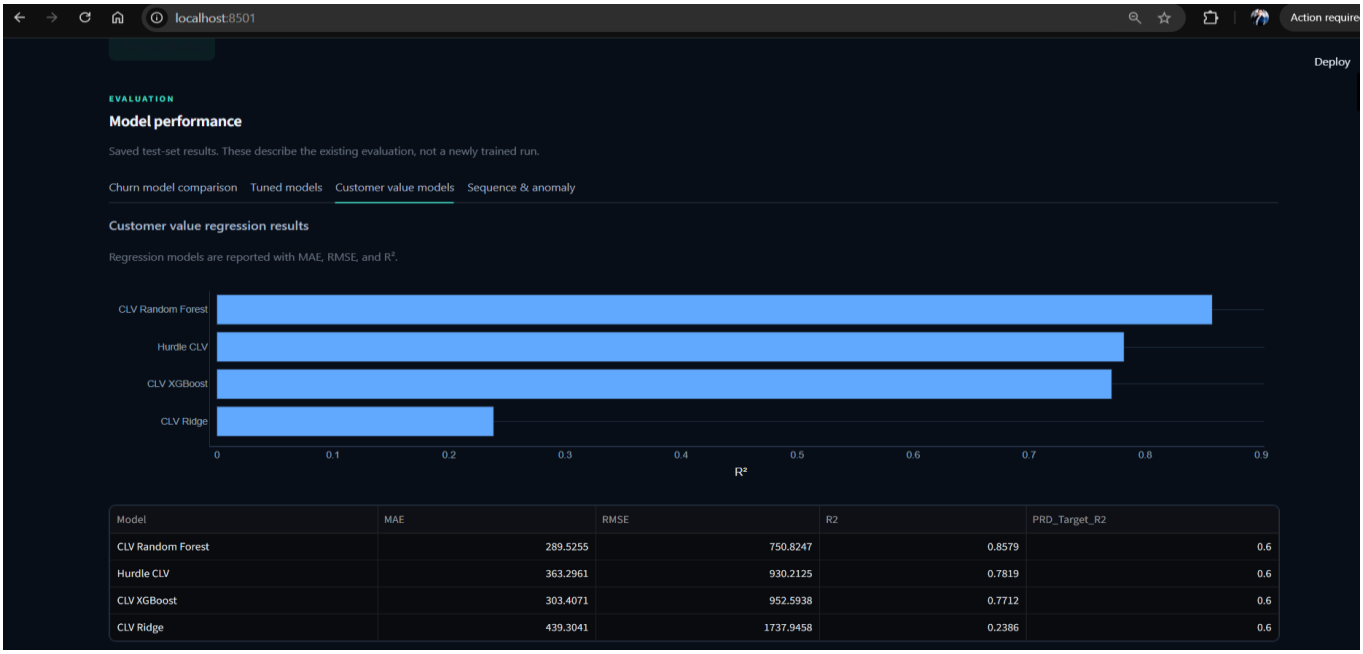




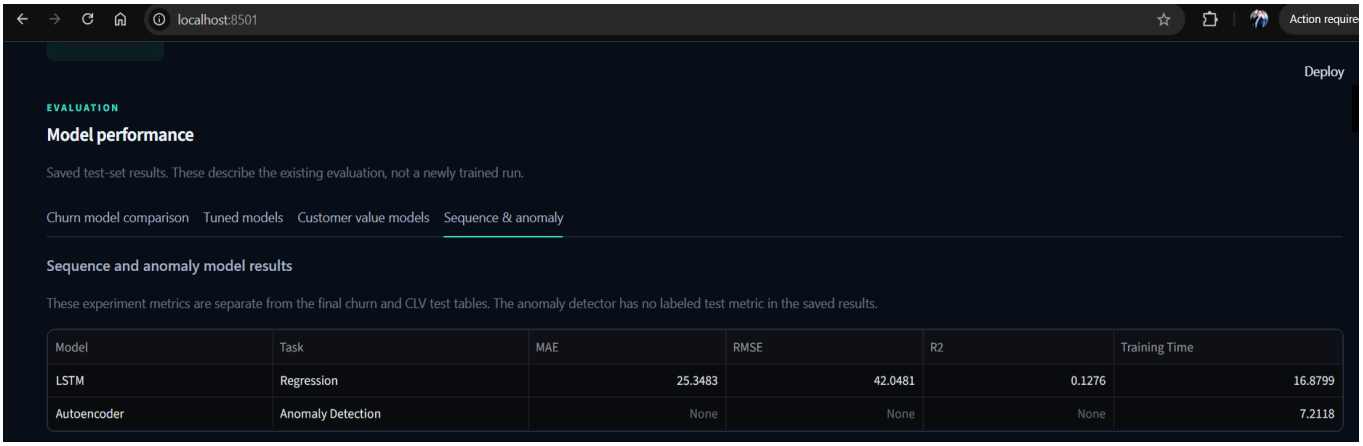
Model	Dataset	Accuracy	Precision	Recall	F1	ROC-AUC	Evaluation Time
Logistic Regression	Test	72.38%	77%	73.05%	74.97%	78.63%	0.0242
Decision Tree	Test	70.87%	73.29%	76.39%	74.81%	76.17%	0.005
Random Forest	Test	73.14%	76.22%	76.39%	76.31%	79.05%	0.051
XGBoost	Test	72.64%	72.83%	82.41%	77.32%	78.7%	0.0302
LightGBM	Test	74.15%	74.21%	83.3%	78.49%	78.91%	0.0127
SVM	Test	72.38%	72.82%	81.74%	77.02%	78.12%	0.0923
ANN	Test	73.52%	73.2%	83.96%	78.22%	78.36%	0.0056
ANN (Keras)	Test	72.26%	70.71%	87.08%	78.04%	78.64%	0.3521



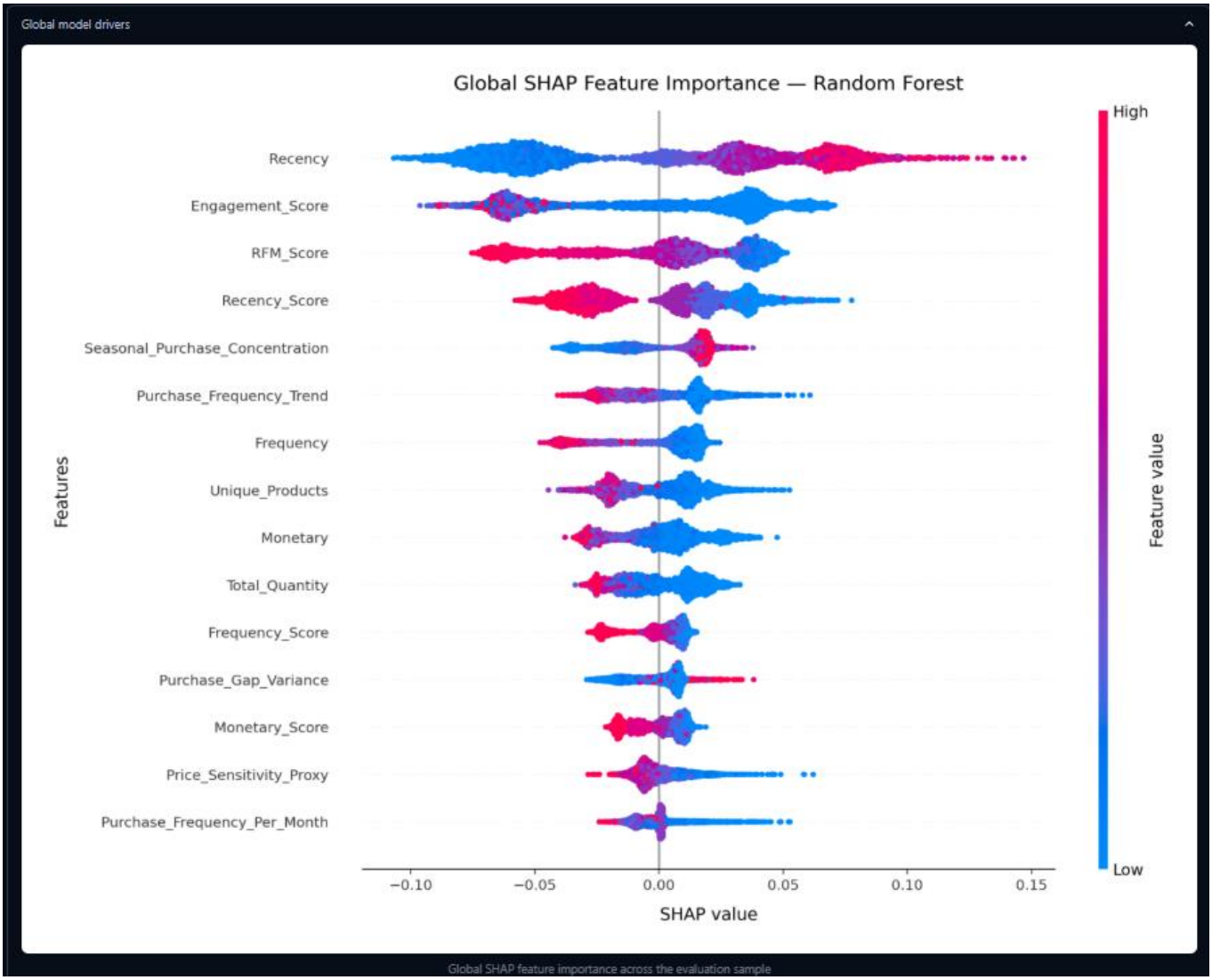
Model	Accuracy	Precision	Recall	F1	ROC-AUC	Evaluation_Time
Random Forest Tuned	74.02%	73.96%	83.52%	78.45%	79.18%	0.0561
XGBoost Tuned	72.51%	72.6%	82.63%	77.29%	78.85%	0.0091
LightGBM Tuned	72.89%	73.21%	82.18%	77.44%	78.6%	0.008



Model	MAE	RMSE	R2	PRD_Target_R2
CLV Random Forest	289.5255	750.8247	0.8579	0.6
Hurdle CLV	363.2961	930.2125	0.7819	0.6
CLV XGBoost	303.4071	952.5938	0.7712	0.6
CLV Ridge	439.3041	1737.9458	0.2386	0.6



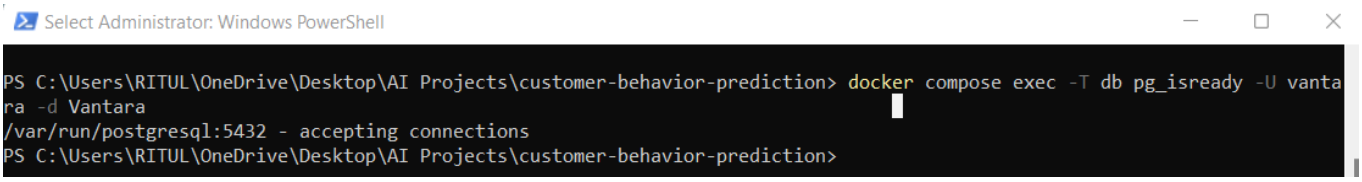
Model	Task	MAE	RMSE	R2	Training Time
LSTM	Regression	25.3483	42.0481	0.1276	16.8799
Autoencoder	Anomaly Detection	None	None	None	7.2118


Global model drivers
Global SHAP Feature Importance — Random Forest
Recency
Engagement_Score
RFM_Score
Recency_Score
Seasonal_Purchase_Concentration
Purchase_Frequency_Trend
Frequency
Unique_Products
Monetary
Total_Quantity
Frequency_Score
Purchase_Gap_Variance
Monetary_Score
Price_Sensitivity_Proxy
Purchase_Frequency_Per_Month
Features
−0.10
−0.05
0.00
0.05
0.10
0.15
SHAP value
High
Low
Feature value
Global SHAP feature importance across the evaluation sample


localhost:8501
Deploy
Purchase_Gap_Variance
Monetary_Score
Price_Sensitivity_Proxy
Purchase_Frequency_Per_Month
−0.10
−0.05
0.00
0.05
0.10
0.15
SHAP value
Low
Global SHAP feature importance across the evaluation sample
Currency is GBP (£), matching the Online Retail II transaction data. Revenue forecasting and product suggestions are baseline methods.
Vantara Customer Intelligence Platform
Created by Ritul Gaikwad
Customer Analytics · Machine Learning · Explainable AI

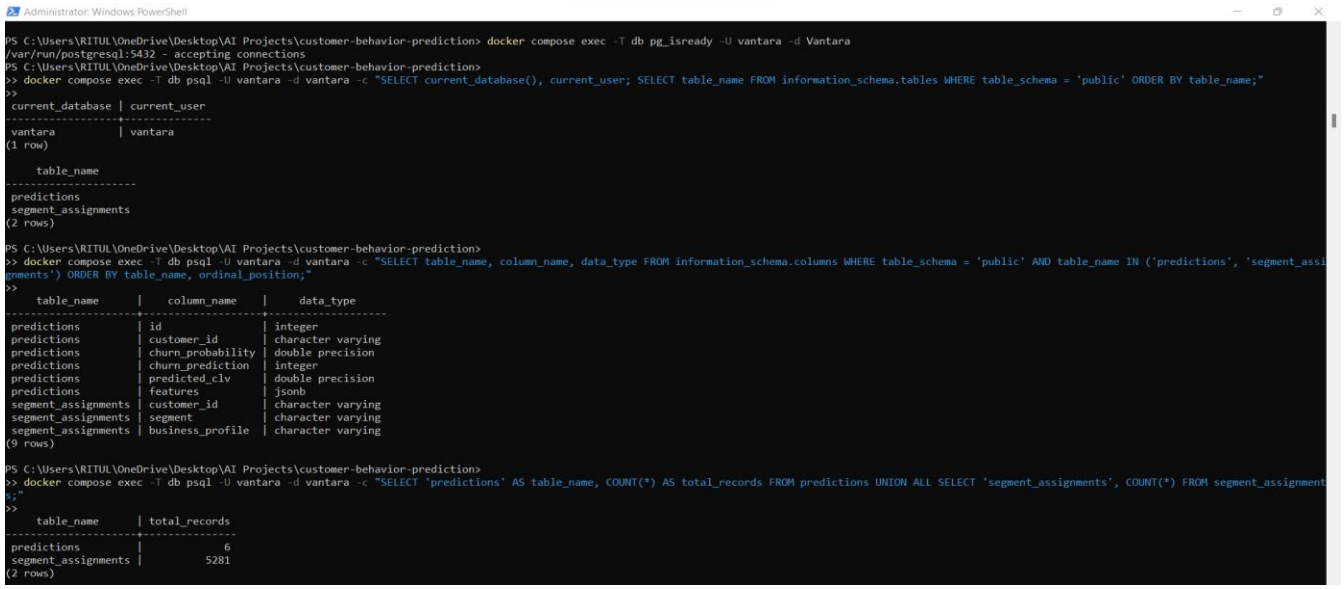
11. Check the Database Service

docker compose exec -T db pg_isready -U vantara -d Vantara



Verification Checklist:

- Compose database service is named db — the command successfully executed against it.
- PostgreSQL is accepting connections.
- Username vantara and database name Vantara were accepted by the readiness check.
- Database readiness verification is complete.



❖ **Database Schema: Predictions and Customer Segmentation**

- **predictions** — Intended to hold customer prediction records, such as churn predictions and predicted customer lifetime value (CLV).
- **segment_assignments** — Intended to store customer segmentation results.

○ Appendix B: Dataset Source and Attribution

Dataset: Online Retail II
Source: UCI Machine Learning Repository
Official Dataset Page: https://archive.ics.uci.edu/dataset/502/online+retail+ii

The project uses the publicly available Online Retail II dataset, which contains transaction-level records from a UK-based online retailer spanning 1 December 2009 to 9 December 2011. The dataset supports data preprocessing, customer-level feature engineering, churn analysis, customer lifetime value estimation, and purchasing-behavior analysis.

Dataset Citation:
Chen, D. (2012). Online Retail II [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5CG6D

License: *Creative Commons Attribution 4.0 International (CC BY 4.0).*

Appendix C: Technical References

- ***Scikit-learn***: *https://scikit-learn.org/stable/*
- ***SHAP***: *https://shap.readthedocs.io/*
- ***FastAPI***: *https://fastapi.tiangolo.com/*
- ***Streamlit***: *https://docs.streamlit.io/*
- ***PostgreSQL***: *https://www.postgresql.org/docs/*
- ***Docker Compose***: *https://docs.docker.com/compose/*
- ***pytest***: *https://docs.pytest.org/*

These references document the technologies used in the project.

— End of Document —